Контест 10 · Данные и метрики

Неделя 20 · темы недель 19–20: события и метрики игры, когорты и удержание, оконные функции SQL, A/B-тесты и честные выводы. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею.

Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

A. Агрегаты

A1. `percentile(xs, p)` — перцентиль с линейной интерполяцией, как в pandas: сортируем, позиция `pos = (n − 1) · p / 100`, между соседними элементами — по прямой. Округлить до 2. Пустой список или `p` вне `0..100` — `ValueError`. Почему время ответа сервера смотрят по p90, а не по среднему?

```
Ввод:  percentile([50, 55, 60, 62, 2400], 90), percentile([10, 20, 30, 40], 50)
Вывод: 1464.8, 25.0
```

A2. `moving_average(xs, k)` — скользящее среднее последних `k` значений, округлить до 2; первые `k − 1` позиций — `None`. Один проход: сумму окна не пересчитываем заново.

```
Ввод:  moving_average([3, 5, 4, 8, 10], 3)
Вывод: [None, None, 4.0, 5.67, 7.33]
```

A3. `histogram(xs, lo, hi, bins)` — `bins` корзин равной ширины `w` на `[lo, hi]`: корзина `i` — `[lo + i·w, lo + (i+1)·w)`, последняя включает `hi`. Значения вне `[lo, hi]` не считаем.

```
Ввод:  histogram([0, 1, 2, 2.5, 5, 9.99, 10, 11, -1], 0, 10, 4)
Вывод: [3, 1, 1, 2]
```

A4*. `welford(xs)` — `xs` может быть генератором: один проход, память O(1). Ответ — `(n, среднее, выборочное стандартное отклонение)`, округлить до 4; при `n < 2` отклонение `0.0`. Метод Уэлфорда: `d = x − mean`, `mean += d / n`, `m2 += d · (x − mean)`. Почему формула «среднее квадратов минус квадрат среднего» врёт на `1e9 + 4, 1e9 + 7, …`?

```
Ввод:  welford([2, 4, 4, 4, 5, 5, 7, 9]), welford([])
Вывод: (8, 5.0, 2.1381), (0, 0.0, 0.0)
```

B. Время и когорты

B1. `to_local_day(ts, offset_hours)` — время события в UTC (`"…Z"`) и пояс игрока → его локальная дата.

```
Ввод:  to_local_day("2027-03-01T22:30:00Z", 3), to_local_day("2027-03-01T02:00:00Z", -5)
Вывод: "2027-03-02", "2027-02-28"
```

B2. `dau_series(events, start, end)` — события `(user, "YYYY-MM-DD")` → число разных игроков в каждый день от `start` до `end` включительно. Дни без событий — `0`, а не пропуск.

```
Ввод:  ev = [(1, "2027-03-01"), (2, "2027-03-01"), (1, "2027-03-01"),
             (1, "2027-03-02"), (3, "2027-03-04")]
       dau_series(ev, "2027-03-01", "2027-03-05")
Вывод: [2, 1, 0, 1, 0]
```

B3. `retention_matrix(events, max_day)` — когорта игрока — день его первого события. Для `N = 0..max_day` — доля когорты, у которой есть событие в день `когорта + N`, округлить до 2. День позже последнего дня в данных — `None` (ещё не наступил — это не ноль). Ключи — даты когорт по возрастанию.

```
Ввод:  ev = [(1, "2027-03-01"), (2, "2027-03-01"), (1, "2027-03-02"), (2, "2027-03-03"),
             (3, "2027-03-02"), (3, "2027-03-03"), (1, "2027-03-03")]
       retention_matrix(ev, 2)
Вывод: {"2027-03-01": [1.0, 0.5, 1.0], "2027-03-02": [1.0, 1.0, None]}
```

C. Как в SQL

C1. `ranks(scores)` — места по убыванию очков для каждого элемента в исходном порядке: пара списков `(RANK, DENSE_RANK)`. У `RANK` после двух вторых мест идёт четвёртое, у `DENSE_RANK` — третье.

```
Ввод:  ranks([300, 250, 250, 100, 300, 90])
Вывод: ([1, 3, 3, 5, 1, 6], [1, 2, 2, 3, 1, 4])
```

C2. `running_by_user(rows)` — аналог `SUM(value) OVER (PARTITION BY user ORDER BY day)`. Строки `(user, day, value)`, дни у одного игрока разные. Ответ — накопленная сумма для каждой строки в исходном порядке строк.

```
Ввод:  running_by_user([("ann", "03-02", 5), ("bob", "03-01", 2), ("ann", "03-01", 3),
                        ("bob", "03-03", 4), ("ann", "03-03", 1)])
Вывод: [8, 2, 3, 6, 9]
```

C3. `pivot(rows)` — сводная таблица: `(строка, столбец, число)` → сумма в каждой клетке. Строки и столбцы по алфавиту, пустая клетка — `0`. Ответ — `(столбцы, [(строка, [значения]), …])`.

```
Ввод:  pivot([("ann", "win", 2), ("bob", "loss", 1), ("ann", "loss", 3),
              ("ann", "win", 1), ("cat", "draw", 1)])
Вывод: (["draw", "loss", "win"],
        [("ann", [0, 3, 3]), ("bob", [0, 1, 0]), ("cat", [1, 0, 0])])
```

C4*. `sessionize(events, gap)` — события `(user, ts)` (секунды, в любом порядке) → сессии каждого игрока: новая сессия начинается, если пауза **больше** `gap`. Сессия — `(начало, конец, событий)`. Ключи — по возрастанию `user`. Как сделать то же самое одним SQL-запросом через `LAG`?

```
Ввод:  ev = [(1, 100), (1, 0), (1, 250), (2, 10), (1, 1000), (1, 1100), (2, 5000)]
       sessionize(ev, 300)
Вывод: {1: [(0, 250, 3), (1000, 1100, 2)], 2: [(10, 10, 1), (5000, 5000, 1)]}
```

D. Честные выводы

D1. `binom_p(k, n)` — точный двусторонний тест «первый ход не даёт преимущества»: `k` побед первого игрока из `n` партий. `p = min(1, 2 · P(X ≤ min(k, n − k)))`, `X` — биномиальное с `p = 1/2` (`math.comb`), округлить до 4. Нужно `0 ≤ k ≤ n`, иначе `ValueError`; `n = 0` — `1.0`.

```
Ввод:  binom_p(9, 10), binom_p(2, 3), binom_p(540, 1000)
Вывод: 0.0215, 1.0, 0.0124
```

D2. `ab_test(ca, na, cb, nb)` — z-тест двух долей: `p = (ca + cb) / (na + nb)`, `se = √(p(1 − p)(1/na + 1/nb))`, `z = (cb/nb − ca/na) / se`, `p_value = erfc(|z| / √2)`. Ответ — `(z, p_value)`, округлить до 4; `se = 0` — `(0.0, 1.0)`. Что значит `p_value = 0.0496` — и чего он **не** значит?

```
Ввод:  ab_test(120, 1000, 150, 1000), ab_test(5, 50, 9, 50)
Вывод: (1.963, 0.0496), (1.1528, 0.249)
```

D3. `simpson(groups)` — группы игроков: имя → `(побед A, партий A, побед B, партий B)`. Кто лучше по доле побед в каждой группе и в сумме: `"A"`, `"B"` или `"tie"` (доли сравнивать перекрёстным умножением, без округления). `paradox` — в сумме выигрывает одна сторона, а во **всех** группах — другая.

```
Ввод:  simpson({"новички": (81, 87, 234, 270), "опытные": (192, 263, 55, 80)})
Вывод: {"overall": "B", "by_group": {"новички": "A", "опытные": "A"}, "paradox": True}
```

E. Проект и алгоритмы

E1. Дашборд вашей игры (семинар 20): три метрики с определениями «что считаем событием» и график DAU с осью от нуля. Объясните одно число, которое вас удивило.

E2. SQL-запрос, который написал агент: запустите его на маленькой базе, где ответ известен заранее, и покажите, что он верен (или найдите ошибку).

E3. Баланс вашей игры: доля побед первого игрока, `binom_p` по ней и вывод — менять ли правила.

E4 · босс. A/B-эксперимент от распределения до решения. За каждую часть свой плюс.

а) `assign(user_id, experiment, variants, weights)` — детерминированное распределение: `h = sha256(f"{experiment}:{user_id}")`, `bucket = int(первые 8 hex-символов, 16) % 100`. Веса в процентах (сумма 100, иначе `ValueError`) делят `0..99` на отрезки по порядку. Зачем в хэше имя эксперимента?

```
Ввод:  [assign(u, "hint_v2", ["control", "test"], [50, 50]) for u in range(1, 5)]
       assign(42, "hint_v2", ["a", "b", "c"], [20, 30, 50])
Вывод: ["test", "control", "test", "test"]
       "c"
```

б) `srm_check(counts, weights)` — sample ratio mismatch: группы не того размера — эксперимент сломан. `χ² = Σ (наблюдали − ждали)² / ждали`, `ждали = всего · вес / 100`; `df = вариантов − 1`. Хвост: при `df = 1` — `erfc(√(χ²/2))`, при `df = 2` — `exp(−χ²/2)`. Ответ — p-value, округлить до 4.

```
Ввод:  srm_check([5000, 5000], [50, 50]), srm_check([5150, 4850], [50, 50])
       srm_check([2100, 2900, 5000], [20, 30, 50])
Вывод: 1.0, 0.0027
       0.0155
```

в) `report(users, converted, experiment, weights=(50, 50))` — варианты `"control"` и `"test"`. Распределить через а), посчитать `n` и `conv` по вариантам, `srm_p` через б), `z` и `p` через D2 (control — A), `lift = доля test − доля control` (округлить до 4). `decision`: `srm_p < 0.01` → `"broken"`; иначе при `p < 0.05` — `"ship"`, если `lift > 0`, и `"rollback"`, если `lift < 0`; иначе `"wait"`.

```
Ввод:  t = lambda u: assign(u, "hint_v2", ["control", "test"], [50, 50]) == "test"
       users = range(1, 2001)
       conv = {u for u in users if u % 5 == 0 or (t(u) and u % 10 == 3)}
       report(users, conv, "hint_v2")
Вывод: {"n": {"control": 982, "test": 1018}, "conv": {"control": 204, "test": 291},
        "srm_p": 0.4208, "z": 4.0468, "p": 0.0001, "lift": 0.0781, "decision": "ship"}
```

E5*. `misra_gries(stream, k)` — частые элементы потока, который не помещается в память: не больше `k - 1` счётчиков. Элемент уже есть — `+1`; есть свободное место — новый счётчик `1`; иначе все счётчики `-1`, нулевые удалить (новый элемент не добавляем). Ответ — счётчики по убыванию значения, при равенстве — по ключу. `heavy_hitters(stream, k)` — вторым проходом оставить кандидатов, встречающихся больше `n / k` раз (по возрастанию). Почему настоящий частый элемент не может потеряться?

```
Ввод:  misra_gries("abracadabraalakazam", 3), heavy_hitters("abracadabraalakazam", 3)
       misra_gries("aabbc", 3)
Вывод: {"a": 4}, ["a"]
       {"a": 1, "b": 1}
```